to the *lending* relation. In general, the asset and city data for a branch must appear once for each loan made by that branch.

The repetition of information in our alternative design is undesirable. Repeating information wastes space. Furthermore, it complicates updating the database. Suppose, for example, that the assets of the Perryridge branch change from 1700000 to 1900000. Under our original design, one tuple of the *branch* relation needs to be changed. Under our alternative design, many tuples of the *lending* relation need to be changed. Thus, updates are more costly under the alternative design than under the original design. When we perform the update in the alternative database, we must ensure that *every* tuple pertaining to the Perryridge branch is updated, or else our database will show two different asset values for the Perryridge branch.

That observation is central to understanding why the alternative design is bad. We know that a bank branch has a unique value of assets, so given a branch name we can uniquely identify the assets value. On the other hand, we know that a branch may make many loans, so given a branch name, we cannot uniquely determine a loan number. In other words, we say that the *functional dependency*

$$branch\text{-}name \rightarrow assets$$

holds on *Lending-schema*, but we do not expect the functional dependency *branch-name* $\rightarrow$ *loan-number* to hold. The fact that a branch has a particular value of assets, and the fact that a branch makes a loan are independent, and, as we have seen, these facts are best represented in separate relations. We shall see that we can use functional dependencies to specify formally when a database design is good.

Another problem with the *Lending-schema* design is that we cannot represent directly the information concerning a branch (*branch-name*, *branch-city*, *assets*) unless there exists at least one loan at the branch. This is because tuples in the *lending* relation require values for *loan-number*, *amount*, and *customer-name*.

One solution to this problem is to introduce *null values*, as we did to handle updates through views. Recall, however, that null values are difficult to handle, as we saw in Section 3.3.4. If we are not willing to deal with null values, then we can create the branch information only when the first loan application at that branch is made. Worse, we would have to delete this information when all the loans have been paid. Clearly, this situation is undesirable, since, under our original database design, the branch information would be available regardless of whether or not loans are currently maintained in the branch, and without resorting to null values.

## 7.3 Functional Dependencies

Functional dependencies play a key role in differentiating good database designs from bad database designs. A **functional dependency** is a type of constraint that is a generalization of the notion of *key*, as discussed in Chapters 2 and 3.

### 7.3.1 Basic Concepts

Functional dependencies are constraints on the set of legal relations. They allow us to express facts about the enterprise that we are modeling with our database.

In Chapter 2, we defined the notion of a *superkey* as follows. Let $R$ be a relation schema. A subset $K$ of $R$ is a **superkey** of $R$ if, in any legal relation $r(R)$, for all pairs $t_1$ and $t_2$ of tuples in $r$ such that $t_1 \neq t_2$, then $t_1[K] \neq t_2[K]$. That is, no two tuples in any legal relation $r(R)$ may have the same value on attribute set $K$.

The notion of functional dependency generalizes the notion of superkey. Consider a relation schema $R$, and let $\alpha \subseteq R$ and $\beta \subseteq R$. The **functional dependency**

$$\alpha \rightarrow \beta$$

holds on schema $R$ if, in any legal relation $r(R)$, for all pairs of tuples $t_1$ and $t_2$ in $r$ such that $t_1[\alpha] = t_2[\alpha]$, it is also the case that $t_1[\beta] = t_2[\beta]$.

Using the functional-dependency notation, we say that $K$ is a superkey of $R$ if $K \rightarrow R$. That is, $K$ is a superkey if, whenever $t_1[K] = t_2[K]$, it is also the case that $t_1[R] = t_2[R]$ (that is, $t_1 = t_2$).

Functional dependencies allow us to express constraints that we cannot express with superkeys. Consider the schema

$$\textit{Loan-info-schema} = (\textit{loan-number}, \textit{branch-name}, \textit{customer-name}, \textit{amount})$$

which is simplification of the *Lending-schema* that we saw earlier. The set of functional dependencies that we expect to hold on this relation schema is

$$\textit{loan-number} \rightarrow \textit{amount}$$
$$\textit{loan-number} \rightarrow \textit{branch-name}$$

We would not, however, expect the functional dependency

$$\textit{loan-number} \rightarrow \textit{customer-name}$$

to hold, since, in general, a given loan can be made to more than one customer (for example, to both members of a husband–wife pair).

We shall use functional dependencies in two ways:

1. To test relations to see whether they are legal under a given set of functional dependencies. If a relation $r$ is legal under a set $F$ of functional dependencies, we say that $r$ **satisfies** $F$.
2. To specify constraints on the set of legal relations. We shall thus concern ourselves with *only* those relations that satisfy a given set of functional dependencies. If we wish to constrain ourselves to relations on schema $R$ that satisfy a set $F$ of functional dependencies, we say that $F$ **holds** on $R$.

Let us consider the relation $r$ of Figure 7.2, to see which functional dependencies are satisfied. Observe that $A \rightarrow C$ is satisfied. There are two tuples that have an $A$ value of $a_1$. These tuples have the same $C$ value—namely, $c_1$. Similarly, the two tuples with an $A$ value of $a_2$ have the same $C$ value, $c_2$. There are no other pairs of distinct tuples that have the same $A$ value. The functional dependency $C \rightarrow A$ is not satisfied, however. To see that it is not, consider the tuples $t_1 = (a_2, b_3, c_2, d_3)$ and

| $A$ | $B$ | $C$ | $D$ |
|---|---|---|---|
| $a_1$ | $b_1$ | $c_1$ | $d_1$ |
| $a_1$ | $b_2$ | $c_1$ | $d_2$ |
| $a_2$ | $b_2$ | $c_2$ | $d_2$ |
| $a_2$ | $b_2$ | $c_2$ | $d_3$ |
| $a_3$ | $b_3$ | $c_2$ | $d_4$ |

**Figure 7.2** Sample relation $r$.

$t_2 = (a_3, b_3, c_2, d_4)$. These two tuples have the same $C$ values, $c_2$, but they have different $A$ values, $a_2$ and $a_3$, respectively. Thus, we have found a pair of tuples $t_1$ and $t_2$ such that $t_1[C] = t_2[C]$, but $t_1[A] \neq t_2[A]$.

Many other functional dependencies are satisfied by $r$, including, for example, the functional dependency $AB \rightarrow D$. Note that we use $AB$ as a shorthand for $\{A,B\}$, to conform with standard practice. Observe that there is no pair of distinct tuples $t_1$ and $t_2$ such that $t_1[AB] = t_2[AB]$. Therefore, if $t_1[AB] = t_2[AB]$, it must be that $t_1 = t_2$ and, thus, $t_1[D] = t_2[D]$. So, $r$ satisfies $AB \rightarrow D$.

Some functional dependencies are said to be **trivial** because they are satisfied by all relations. For example, $A \rightarrow A$ is satisfied by all relations involving attribute $A$. Reading the definition of functional dependency literally, we see that, for all tuples $t_1$ and $t_2$ such that $t_1[A] = t_2[A]$, it is the case that $t_1[A] = t_2[A]$. Similarly, $AB \rightarrow A$ is satisfied by all relations involving attribute $A$. In general, a functional dependency of the form $\alpha \rightarrow \beta$ is **trivial** if $\beta \subseteq \alpha$.

To distinguish between the concepts of a relation satisfying a dependency and a dependency holding on a schema, we return to the banking example. If we consider the *customer* relation (on *Customer-schema*) in Figure 7.3, we see that *customer-street* $\rightarrow$ *customer-city* is satisfied. However, we believe that, in the real world, two cities

| *customer-name* | *customer-street* | *customer-city* |
|---|---|---|
| Jones | Main | Harrison |
| Smith | North | Rye |
| Hayes | Main | Harrison |
| Curry | North | Rye |
| Lindsay | Park | Pittsfield |
| Turner | Putnam | Stamford |
| Williams | Nassau | Princeton |
| Adams | Spring | Pittsfield |
| Johnson | Alma | Palo Alto |
| Glenn | Sand Hill | Woodside |
| Brooks | Senator | Brooklyn |
| Green | Walnut | Stamford |

**Figure 7.3** The *customer* relation.

| *loan-number* | *branch-name* | *amount* |
|---|---|---|
| L-17 | Downtown | 1000 |
| L-23 | Redwood | 2000 |
| L-15 | Perryridge | 1500 |
| L-14 | Downtown | 1500 |
| L-93 | Mianus | 500 |
| L-11 | Round Hill | 900 |
| L-29 | Pownal | 1200 |
| L-16 | North Town | 1300 |
| L-18 | Downtown | 2000 |
| L-25 | Perryridge | 2500 |
| L-10 | Brighton | 2200 |

**Figure 7.4** The *loan* relation.

can have streets with the same name. Thus, it is possible, at some time, to have an instance of the *customer* relation in which *customer-street* → *customer-city* is not satisfied. So, we would not include *customer-street* → *customer-city* in the set of functional dependencies that hold on *Customer-schema*.

In the *loan* relation (on *Loan-schema*) of Figure 7.4, we see that the dependency *loan-number* → *amount* is satisfied. In contrast to the case of *customer-city* and *customer-street* in *Customer-schema*, we do believe that the real-world enterprise that we are modeling requires each loan to have only one amount. Therefore, we want to require that *loan-number* → *amount* be satisfied by the *loan* relation at all times. In other words, we require that the constraint *loan-number* → *amount* hold on *Loan-schema*.

In the *branch* relation of Figure 7.5, we see that *branch-name* → *assets* is satisfied, as is *assets* → *branch-name*. We want to require that *branch-name* → *assets* hold on *Branch-schema*. However, we do not wish to require that *assets* → *branch-name* hold, since it is possible to have several branches that have the same asset value.

In what follows, we assume that, when we design a relational database, we first list those functional dependencies that must always hold. In the banking example, our list of dependencies includes the following:

| *branch-name* | *branch-city* | *assets* |
|---|---|---|
| Downtown | Brooklyn | 9000000 |
| Redwood | Palo Alto | 2100000 |
| Perryridge | Horseneck | 1700000 |
| Mianus | Horseneck | 400000 |
| Round Hill | Horseneck | 8000000 |
| Pownal | Bennington | 300000 |
| North Town | Rye | 3700000 |
| Brighton | Brooklyn | 7100000 |

**Figure 7.5** The *branch* relation.

- On *Branch-schema*:

  *branch-name* → *branch-city*
  *branch-name* → *assets*

- On *Customer-schema*:

  *customer-name* → *customer-city*
  *customer-name* → *customer-street*

- On *Loan-schema*:

  *loan-number* → *amount*
  *loan-number* → *branch-name*

- On *Borrower-schema*:

  No functional dependencies

- On *Account-schema*:

  *account-number* → *branch-name*
  *account-number* → *balance*

- On *Depositor-schema*:

  No functional dependencies

### 7.3.2 Closure of a Set of Functional Dependencies

It is not sufficient to consider the given set of functional dependencies. Rather, we need to consider *all* functional dependencies that hold. We shall see that, given a set $F$ of functional dependencies, we can prove that certain other functional dependencies hold. We say that such functional dependencies are "logically implied" by $F$.

More formally, given a relational schema $R$, a functional dependency $f$ on $R$ is **logically implied** by a set of functional dependencies $F$ on $R$ if every relation instance $r(R)$ that satisfies $F$ also satisfies $f$.

Suppose we are given a relation schema $R = (A, B, C, G, H, I)$ and the set of functional dependencies

$$A \rightarrow B$$
$$A \rightarrow C$$
$$CG \rightarrow H$$
$$CG \rightarrow I$$
$$B \rightarrow H$$

The functional dependency

$$A \rightarrow H$$

is logically implied. That is, we can show that, whenever our given set of functional dependencies holds on a relation, $A \rightarrow H$ must also hold on the relation. Suppose that $t_1$ and $t_2$ are tuples such that

$$t_1[A] = t_2[A]$$

Since we are given that $A \rightarrow B$, it follows from the definition of functional dependency that

$$t_1[B] = t_2[B]$$

Then, since we are given that $B \rightarrow H$, it follows from the definition of functional dependency that

$$t_1[H] = t_2[H]$$

Therefore, we have shown that, whenever $t_1$ and $t_2$ are tuples such that $t_1[A] = t_2[A]$, it must be that $t_1[H] = t_2[H]$. But that is exactly the definition of $A \rightarrow H$.

Let $F$ be a set of functional dependencies. The **closure** of $F$, denoted by $F^+$, is the set of all functional dependencies logically implied by $F$. Given $F$, we can compute $F^+$ directly from the formal definition of functional dependency. If $F$ were large, this process would be lengthy and difficult. Such a computation of $F^+$ requires arguments of the type just used to show that $A \rightarrow H$ is in the closure of our example set of dependencies.

**Axioms**, or rules of inference, provide a simpler technique for reasoning about functional dependencies. In the rules that follow, we use Greek letters ($\alpha$, $\beta$, $\gamma$, ... ) for sets of attributes, and uppercase Roman letters from the beginning of the alphabet for individual attributes. We use $\alpha\beta$ to denote $\alpha \cup \beta$.

We can use the following three rules to find logically implied functional dependencies. By applying these rules *repeatedly*, we can find all of $F^+$, given $F$. This collection of rules is called **Armstrong's axioms** in honor of the person who first proposed it.

- **Reflexivity rule**. If $\alpha$ is a set of attributes and $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ holds.
- **Augmentation rule**. If $\alpha \rightarrow \beta$ holds and $\gamma$ is a set of attributes, then $\gamma\alpha \rightarrow \gamma\beta$ holds.
- **Transitivity rule**. If $\alpha \rightarrow \beta$ holds and $\beta \rightarrow \gamma$ holds, then $\alpha \rightarrow \gamma$ holds.

Armstrong's axioms are **sound**, because they do not generate any incorrect functional dependencies. They are **complete**, because, for a given set $F$ of functional dependencies, they allow us to generate all $F^+$. The bibliographical notes provide references for proofs of soundness and completeness.

Although Armstrong's axioms are complete, it is tiresome to use them directly for the computation of $F^+$. To simplify matters further, we list additional rules. It is possible to use Armstrong's axioms to prove that these rules are correct (see Exercises 7.8, 7.9, and 7.10).

- **Union rule**. If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds.
- **Decomposition rule**. If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
- **Pseudotransitivity rule**. If $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds.

Let us apply our rules to the example of schema $R = (A, B, C, G, H, I)$ and the set $F$ of functional dependencies $\{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$. We list several members of $F^+$ here:

- $A \rightarrow H$. Since $A \rightarrow B$ and $B \rightarrow H$ hold, we apply the transitivity rule. Observe that it was much easier to use Armstrong's axioms to show that $A \rightarrow H$ holds than it was to argue directly from the definitions, as we did earlier in this section.
- $CG \rightarrow HI$. Since $CG \rightarrow H$ and $CG \rightarrow I$, the union rule implies that $CG \rightarrow HI$.
- $AG \rightarrow I$. Since $A \rightarrow C$ and $CG \rightarrow I$, the pseudotransitivity rule implies that $AG \rightarrow I$ holds.

  Another way of finding that $AG \rightarrow I$ holds is as follows. We use the augmentation rule on $A \rightarrow C$ to infer $AG \rightarrow CG$. Applying the transitivity rule to this dependency and $CG \rightarrow I$, we infer $AG \rightarrow I$.

Figure 7.6 shows a procedure that demonstrates formally how to use Armstrong's axioms to compute $F^+$. In this procedure, when a functional dependency is added to $F^+$, it may be already present, and in that case there is no change to $F^+$. We will also see an alternative way of computing $F^+$ in Section 7.3.3.

The left-hand and right-hand sides of a functional dependency are both subsets of $R$. Since a set of size $n$ has $2^n$ subsets, there are a total of $2 \times 2^n = 2^{n+1}$ possible functional dependencies, where $n$ is the number of attributes in $R$. Each iteration of the repeat loop of the procedure, except the last iteration, adds at least one functional dependency to $F^+$. Thus, the procedure is guaranteed to terminate.

## 7.3.3 Closure of Attribute Sets

To test whether a set $\alpha$ is a superkey, we must devise an algorithm for computing the set of attributes functionally determined by $\alpha$. One way of doing this is to compute $F^+$, take all functional dependencies with $\alpha$ as the left-hand side, and take the union of the right-hand sides of all such dependencies. However, doing so can be expensive, since $F^+$ can be large.

```
F+ = F
repeat
    for each functional dependency f in F+
        apply reflexivity and augmentation rules on f
        add the resulting functional dependencies to F+
    for each pair of functional dependencies f1 and f2 in F+
        if f1 and f2 can be combined using transitivity
            add the resulting functional dependency to F+
until F+ does not change any further
```

**Figure 7.6** A procedure to compute $F^+$.

An efficient algorithm for computing the set of attributes functionally determined by $\alpha$ is useful not only for testing whether $\alpha$ is a superkey, but also for several other tasks, as we will see later in this section.

Let $\alpha$ be a set of attributes. We call the set of all attributes functionally determined by $\alpha$ under a set $F$ of functional dependencies the **closure** of $\alpha$ under $F$; we denote it by $\alpha^+$. Figure 7.7 shows an algorithm, written in pseudocode, to compute $\alpha^+$. The input is a set $F$ of functional dependencies and the set $\alpha$ of attributes. The output is stored in the variable *result*.

To illustrate how the algorithm works, we shall use it to compute $(AG)^+$ with the functional dependencies defined in Section 7.3.2. We start with $result = AG$. The first time that we execute the **while** loop to test each functional dependency, we find that

- $A \rightarrow B$ causes us to include $B$ in *result*. To see this fact, we observe that $A \rightarrow B$ is in $F$, $A \subseteq result$ (which is $AG$), so $result := result \cup B$.
- $A \rightarrow C$ causes *result* to become *ABCG*.
- $CG \rightarrow H$ causes *result* to become *ABCGH*.
- $CG \rightarrow I$ causes *result* to become *ABCGHI*.

The second time that we execute the **while** loop, no new attributes are added to *result*, and the algorithm terminates.

Let us see why the algorithm of Figure 7.7 is correct. The first step is correct, since $\alpha \rightarrow \alpha$ always holds (by the reflexivity rule). We claim that, for any subset $\beta$ of *result*, $\alpha \rightarrow \beta$. Since we start the **while** loop with $\alpha \rightarrow result$ being true, we can add $\gamma$ to *result* only if $\beta \subseteq result$ and $\beta \rightarrow \gamma$. But then $result \rightarrow \beta$ by the reflexivity rule, so $\alpha \rightarrow \beta$ by transitivity. Another application of transitivity shows that $\alpha \rightarrow \gamma$ (using $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$). The union rule implies that $\alpha \rightarrow result \cup \gamma$, so $\alpha$ functionally determines any new result generated in the **while** loop. Thus, any attribute returned by the algorithm is in $\alpha^+$.

It is easy to see that the algorithm finds all $\alpha^+$. If there is an attribute in $\alpha^+$ that is not yet in *result*, then there must be a functional dependency $\beta \rightarrow \gamma$ for which $\beta \subseteq result$, and at least one attribute in $\gamma$ is not in *result*.

It turns out that, in the worst case, this algorithm may take an amount of time quadratic in the size of $F$. There is a faster (although slightly more complex) algorithm that runs in time linear in the size of $F$; that algorithm is presented as part of Exercise 7.14.

```
result := α;
while (changes to result) do
        for each functional dependency β → γ in F do
            begin
                if β ⊆ result then result := result ∪ γ;
            end
```

**Figure 7.7** An algorithm to compute $\alpha^+$, the closure of $\alpha$ under $F$.

There are several uses of the attribute closure algorithm:

- To test if $\alpha$ is a superkey, we compute $\alpha^+$, and check if $\alpha^+$ contains all attributes of $R$.
- We can check if a functional dependency $\alpha \rightarrow \beta$ holds (or, in other words, is in $F^+$), by checking if $\beta \subseteq \alpha^+$. That is, we compute $\alpha^+$ by using attribute closure, and then check if it contains $\beta$. This test is particularly useful, as we will see later in this chapter.
- It gives us an alternative way to compute $F^+$: For each $\gamma \subseteq R$, we find the closure $\gamma^+$, and for each $S \subseteq \gamma^+$, we output a functional dependency $\gamma \rightarrow S$.

### 7.3.4 Canonical Cover

Suppose that we have a set of functional dependencies $F$ on a relation schema. Whenever a user performs an update on the relation, the database system must ensure that the update does not violate any functional dependencies, that is, all the functional dependencies in $F$ are satisfied in the new database state.

The system must roll back the update if it violates any functional dependencies in the set $F$.

We can reduce the effort spent in checking for violations by testing a simplified set of functional dependencies that has the same closure as the given set. Any database that satisfies the simplified set of functional dependencies will also satisfy the original set, and vice versa, since the two sets have the same closure. However, the simplified set is easier to test. We shall see how the simplified set can be constructed in a moment. First, we need some definitions.

An attribute of a functional dependency is said to be **extraneous** if we can remove it without changing the closure of the set of functional dependencies. The formal definition of **extraneous attributes** is as follows. Consider a set $F$ of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in $F$.

- Attribute $A$ is extraneous in $\alpha$ if $A \in \alpha$, and $F$ logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- Attribute $A$ is extraneous in $\beta$ if $A \in \beta$, and the set of functional dependencies $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies $F$.

For example, suppose we have the functional dependencies $AB \rightarrow C$ and $A \rightarrow C$ in $F$. Then, $B$ is extraneous in $AB \rightarrow C$. As another example, suppose we have the functional dependencies $AB \rightarrow CD$ and $A \rightarrow C$ in $F$. Then $C$ would be extraneous in the right-hand side of $AB \rightarrow CD$.

Beware of the direction of the implications when using the definition of extraneous attributes: If you exchange the left-hand side with right-hand side, the implication will *always* hold. That is, $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$ always logically implies $F$, and also $F$ always logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$

Here is how we can test efficiently if an attribute is extraneous. Let $R$ be the relation schema, and let $F$ be the given set of functional dependencies that hold on $R$. Consider an attribute $A$ in a dependency $\alpha \rightarrow \beta$.

- If $A \in \beta$, to check if $A$ is extraneous consider the set
  $$F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$$
  and check if $\alpha \rightarrow A$ can be inferred from $F'$. To do so, compute $\alpha^+$ (the closure of $\alpha$) under $F'$; if $\alpha^+$ includes $A$, then $A$ is extraneous in $\beta$.
- If $A \in \alpha$, to check if $A$ is extraneous, let $\gamma = \alpha - \{A\}$, and check if $\gamma \rightarrow \beta$ can be inferred from $F$. To do so, compute $\gamma^+$ (the closure of $\gamma$) under $F$; if $\gamma^+$ includes all attributes in $\beta$, then $A$ is extraneous in $\alpha$.

For example, suppose $F$ contains $AB \rightarrow CD$, $A \rightarrow E$, and $E \rightarrow C$. To check if $C$ is extraneous in $AB \rightarrow CD$, we compute the attribute closure of $AB$ under $F' = \{AB \rightarrow D, A \rightarrow E, \text{and } E \rightarrow C\}$. The closure is $ABCDE$, which includes $CD$, so we infer that $C$ is extraneous.

A **canonical cover** $F_c$ for $F$ is a set of dependencies such that $F$ logically implies all dependencies in $F_c$, and $F_c$ logically implies all dependencies in $F$. Furthermore, $F_c$ must have the following properties:

- No functional dependency in $F_c$ contains an extraneous attribute.
- Each left side of a functional dependency in $F_c$ is unique. That is, there are no two dependencies $\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ in $F_c$ such that $\alpha_1 = \alpha_2$.

A canonical cover for a set of functional dependencies $F$ can be computed as depicted in Figure 7.8. It is important to note that when checking if an attribute is extraneous, the check uses the dependencies in the current value of $F_c$, and **not** the dependencies in $F$. If a functional dependency contains only one attribute in its right-hand side, for example $A \rightarrow C$, and that attribute is found to be extraneous, we would get a functional dependency with an empty right-hand side. Such functional dependencies should be deleted.

The canonical cover of $F$, $F_c$, can be shown to have the same closure as $F$; hence, testing whether $F_c$ is satisfied is equivalent to testing whether $F$ is satisfied. However, $F_c$ is minimal in a certain sense—it does not contain extraneous attributes, and it

$F_c = F$
**repeat**
  Use the union rule to replace any dependencies in $F_c$ of the form
    $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1 \beta_2$.
  Find a functional dependency $\alpha \rightarrow \beta$ in $F_c$ with an extraneous
    attribute either in $\alpha$ or in $\beta$.
    /* Note: the test for extraneous attributes is done using $F_c$, not $F$ */
  If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$.
**until** $F_c$ does not change.

**Figure 7.8** Computing canonical cover

combines functional dependencies with the same left side. It is cheaper to test $F_c$ than it is to test $F$ itself.

Consider the following set $F$ of functional dependencies on schema $(A, B, C)$:

$$A \rightarrow BC$$
$$B \rightarrow C$$
$$A \rightarrow B$$
$$AB \rightarrow C$$

Let us compute the canonical cover for $F$.

- There are two functional dependencies with the same set of attributes on the left side of the arrow:

  $$A \rightarrow BC$$
  $$A \rightarrow B$$

  We combine these functional dependencies into $A \rightarrow BC$.

- $A$ is extraneous in $AB \rightarrow C$ because $F$ logically implies $(F - \{AB \rightarrow C\}) \cup \{B \rightarrow C\}$. This assertion is true because $B \rightarrow C$ is already in our set of functional dependencies.

- $C$ is extraneous in $A \rightarrow BC$, since $A \rightarrow BC$ is logically implied by $A \rightarrow B$ and $B \rightarrow C$.

Thus, our canonical cover is

$$A \rightarrow B$$
$$B \rightarrow C$$

Given a set $F$ of functional dependencies, it may be that an entire functional dependency in the set is extraneous, in the sense that dropping it does not change the closure of $F$. We can show that a canonical cover $F_c$ of $F$ contains no such extraneous functional dependency. Suppose that, to the contrary, there were such an extraneous functional dependency in $F_c$. The right-side attributes of the dependency would then be extraneous, which is not possible by the definition of canonical covers.

A canonical cover might not be unique. For instance, consider the set of functional dependencies $F = \{A \rightarrow BC, B \rightarrow AC$, and $C \rightarrow AB\}$. If we apply the extraneity test to $A \rightarrow BC$, we find that both $B$ and $C$ are extraneous under $F$. However, it is incorrect to delete both! The algorithm for finding the canonical cover picks one of the two, and deletes it. Then,

1. If $C$ is deleted, we get the set $F' = \{A \rightarrow B, B \rightarrow AC$, and $C \rightarrow AB\}$. Now, $B$ is not extraneous in the righthand side of $A \rightarrow B$ under $F'$. Continuing the algorithm, we find $A$ and $B$ are extraneous in the right-hand side of $C \rightarrow AB$, leading to two canonical covers

   $$F_c = \{A \rightarrow B, B \rightarrow C, \text{and } C \rightarrow A\}, \text{ and}$$
   $$F_c = \{A \rightarrow B, B \rightarrow AC, \text{and } C \rightarrow B\}.$$